

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**ПРЕВРЪЩАНЕ НА ВЪПРОСИ НА ЕСТЕСТВЕН
ЕЗИК ВЪВ ФОРМАЛНИ ЗАЯВКИ:
МОРФОЛОГИЧЕН, СИНТАКТИЧЕН И
СЕМАНТИЧЕН АНАЛИЗ СЪС СИНТЕЗ НА
ПОТВЪРЖДАВАЩО ИЗРЕЧЕНИЕ**

КУРСОВА РАБОТА

по дисциплина „Интерфейс на естествен език“

Разработил:

Алекс Цветанов, фак. № 121326006

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Доц. д-р Диана Григорова

София, 2027

СЪДЪРЖАНИЕ

УВОД	1
I. ТЕОРЕТИЧНА ОСНОВА И ОБОСНОВКА НА ИЗБОРА	2
II. ПРОЕКТИРАНЕ НА СИСТЕМАТА	3
2.1. Структура на интерфейса	3
2.2. Морфологичен анализ	3
2.3. Синтактичен анализ	4
2.4. Семантичен анализ	5
2.5. Генериране на формална заявка	5
2.6. Синтез на изречение	5
III. ПРОГРАМНА РЕАЛИЗАЦИЯ	6
IV. МЕТОДИКА И ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ	7
4.1. Какво се мери и как	7
4.2. Работата на конвейера върху един въпрос	7
4.3. Покритие и съгласуваност по слоеве	8
4.4. Поведение при нееднозначност	9
4.5. Нееднозначност спрямо размера на граматиката	10
4.6. Време за анализ	11
4.7. Съпоставка с базово решение и анализ на провалите	11
ЗАКЛЮЧЕНИЕ	15
ЛИТЕРАТУРА	17

УВОД

Достъпът до структурирани данни изисква владение на формален език за заявки. Човек, който знае какво търси в една база от данни, често не знае как да го напише на SQL. Интерфейсът на естествен език премахва точно тази бариера.

Настоящата курсова работа разработва такъв интерфейс под името *Lexicon*. Системата приема въпрос на естествен език, анализира го морфологично, синтактично и семантично, и генерира заявка на SQL или SPARQL срещу зададена предметна област. Обработката е изцяло символна: всяка стъпка от входното изречение до изходната заявка е представима, проверяема и обяснима.

Целта е работеща верига от анализ и синтез, която превръща въпрос във формална заявка и връща на потребителя перифраза на разбрания въпрос, за да може той да потвърди или отхвърли тълкуването, преди заявката да бъде изпълнена. Оттук произтичат задачите: морфологичен анализатор върху краен преобразувател, синтактичен анализатор, който запазва всички разчитания, семантично изображение върху модел на знанието, генератор на формална заявка, синтез на потвърждаваща перифраза и методика за експериментална проверка.

Обхватът е ограничен до въпросителни изречения от предметната област, описана в лексикона и в модела на знанието. Извън обхвата остават свободен диалог, анафора през няколко реплики и произволна предметна област без предварително описание.

Раздел I излага теоретичната основа и обосновава избора за всеки слой. Раздел II представя проектирането, Раздел III програмната реализация, а Раздел IV методиката за измерване и получените резултати.

I. ТЕОРЕТИЧНА ОСНОВА И ОБОСНОВКА НА ИЗБОРА

Три свойства на естествения език поражда целия инженерен проблем. *Нееднозначността*: една и съща поредица от думи допуска повече от едно разчитане на всяко ниво. *Продуктивността*: множеството от допустими изречения е безкрайно, тоест списък от разпознати въпроси не решава задачата. *Контекстната зависимост*: значението зависи от предметната област. Флективността на българския мести тежестта на анализа върху морфологията.

Математическото описание тръгва от йерархията на формалните граматики и от въпроса къде в нея попада естественият език [1]. Практическият избор е безконтекстна граматика с признаци, за която има ефективни алгоритми [2, 3]. На морфологично ниво апаратът е този на крайните преобразуватели [4]: едно устройство изобразява словоформа върху лексикална форма и обратно, тоест служи и за анализ, и за синтез. От психолингвистиката се пренасят две свойства, а не цяла теория: инкременталността, която е свойство и на Earley, и предпочитанието за близко прикрепване, което тук избира между разчитанията; мрежите с преходи [5] са ранният опит двете да се съчетаят. Крайните автомати за много образци едновременно са предмет на публикация на автора [6]. Прегледаните символни интерфейси [7] са обясними по построение, обучените върху двойки [8, 9] не са.

Изборът по слоеве се прави по пет критерия: покритие, поведение при нееднозначност, обяснимост, цена на пренасяне и сложност.

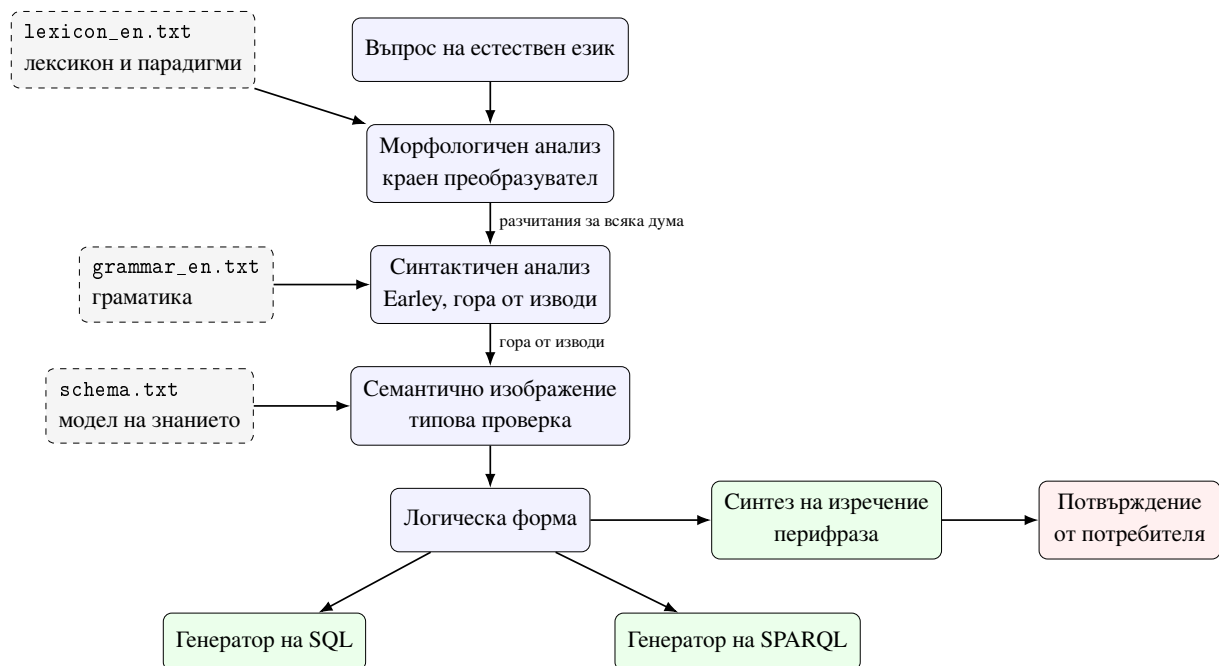
- **Морфология:** преобразувател, а не списък от словоформи: списъкът расте линейно с лексикона и не обработва непозната дума.
- **Синтактичен анализ:** Earley [2], а не CYK [3], защото CYK изисква нормална форма на Чомски, която изкривява дървото на извода, а изобразението е закачено за продукциите както са написани. CYK не е реализиран, затова измерена съпоставка по време няма.
- **Модел на знанието:** междинен. Релационният е близък до SQL, но описва слабо йерархията от понятия; графовият [10] е обратното; фреймовият има вграден извод [11], ненужен тук. Затова логическата форма е отделен слой, от който излизат и двата целеви езика.
- **Голям езиков модел:** базово решение за сравнение, не част от системата: по-широко покритие, но без проверима следа, а изходът изглежда правдоподобен и когато е грешен.

II. ПРОЕКТИРАНЕ НА СИСТЕМАТА

2.1. Структура на интерфейса

Интерфейсът е конвейер от четири анализиращи слоя и един синтезиращ. Входът е низ на естествен език, изходът е формална заявка заедно с перифраза на разбирания въпрос (Фиг. 1).

Изискванията са пет. Всеки слой приема и връща данни по описан договор и може да бъде заменен без промяна в останалите. Неоднозначността се пренася нагоре, вместо да се разрешава преждевременно с евристика на ниско ниво. Всяко тълкуване е проследимо назад до думите, които са го породили. Заявката не се изпълнява преди потвърждение. И накрая, цялото знание за езика и за предметната област живее във файлове с данни, тоест смяната на предметната област е смяна на данни, а не на изходен текст.



Фигура 1. Структура на конвейера и поток на данните между слоевете

2.2. Морфологичен анализ

Слоят връща за всяка словоформа множество от разчитания, всяко с лексема, част на речта и граматични признаци. Множество, а не единично разчитане, защото омонимията е обичайна и се разрешава едва по-нагоре: *films* се анализира и като множествено число на съществителното, и като лична форма на глагола.

Преобразувателят е композиция на две релации: лексиконът, тоест основи с лексема, част на речта и парадигма, и афиксната релация на парадигмата, тоест форми във вида „изтрий n знака от основата, после добави този суфикс“ заедно с признаците, които афиксът внася. Композицията се извършва еднократно при зареждане, дава детерминиран ацикличен преобразувател, в който всеки път изписва една словоформа, и се пази и в обратната посока.

Две решения заслужават коментар. Изтриването в афикса се брои в знаци, а не в байтове: английският лексикон не би забелязал разликата, българският би, защото кирилската буква заема два байта в UTF-8. И редуването живее в данните: `~1+ies` казва „махни един знак, добави `ies`“, а `~6+people` превръща `person` в `people`. Неправилната форма е ред в лексикона, а не нов `C++`.

2.3. Синтактичен анализ

Слоят приема поредица от множества с разчитания и връща гора от изводи. Терминал на граматиката не е дума, а част на речта с ограничение върху признаците, затова граматиката е независима от лексикона.

Елементите на таблицата на Earley се обединяват по ключ „продукция, позиция на точката, начало“ за всяка крайна позиция, а всеки елемент носи списък от връзки. Една връзка е един начин да се стигне до елемента: предшественикът с точка една позиция вляво и възелът, който запълва символа между тях. Неоднозначното изречение затова умножава връзки, а не елементи, и точно това прави гората уплътнена. Възелът се идентифицира с тройката „символ, начало, край“, така че еднаквите подфрази се споделят. Броят на изводите се пресмята върху гората чрез динамично оптимизиране, а не чрез изброяване; граматиката няма празни продукции и няма цикъл от единични продукции, откъдето гората е ациклична и броят е краен.

Неоднозначността по прикрепване живее в точно три продукции: `NOM -> NOM PP`, `VP -> VP PP` и `VPPASS -> VPPASS PP`. Нищо друго не поражда две дървета за един и същ низ, така че отчетената неоднозначност е неоднозначност по прикрепване, а не артефакт от излишна продукция.

2.4. Семантичен анализ

Изображението е композиционно: всяка продукция носи име на семантично правило, а правилото изгражда значението на групата от значенията на съставките ѝ. Тук разчитанията отпадат. Всеки атом се проверява типово спрямо модела, а предлогът се превръща в релация само ако моделът съдържа *рамка* за него, тоест тройката „предлог, тип на аргумента, тип на носителя“ заедно с релацията и реда на аргументите. Затова *in 2010* се прикрепя към филм, а *in Inception* към лице: двата се различават по типа на именната група, а не по правило за думата *in*. Разчитане, което не се изобразява върху нищо в модела, се отхвърля с посочена причина, и това отхвърляне, а не ранна евристика, разрешава нееднозначността.

Когато оцелее повече от едно разчитане, изборът се прави по предпочитание за близко прикрепване: сумата от разстоянията между всяка предложна група и главата, към която се прикрепя, като по-малката сума печели. Предпочитанието се прилага след семантичния филтър, никога преди него. Разчитанията, чиито логически форми съвпадат след канонично преименуване на променливите, се събират в едно, така че показаният брой е брой на значенията, а не на дърветата.

2.5. Генериране на формална заявка

Генераторът работи изцяло върху проверената логическа форма и не съдържа евристики върху текста, затова една и съща семантика се насочва към две цели без промяна в предходните слоеве. Съответствието е пряко: едноместният атом става FROM с псевдоним и тройка с а; двуместният става съединение по таблица или равенство върху колона, срещу една тройка; проекцията става SELECT DISTINCT и в двата. Броенето е COUNT(DISTINCT . . .), въпросът с отговор да или не е EXISTS и ASK, отрицанието е NOT EXISTS и FILTER NOT EXISTS, а всеобщността е релационно деление с две вложени отрицания.

2.6. Синтез на изречение

Синтезът произвежда изречение от логическата форма и го показва като перифраза на разбирания въпрос. Ролята му е контролна: при грешно избрано разчитане потребителят вижда това преди изпълнението. Перифразата се изгражда от формата, не от въпроса; иначе щеше да се съгласи с потребителя по построение.

III. ПРОГРАМНА РЕАЛИЗАЦИЯ

Реализацията е на C++20 и се изгражда с CMake. Разделението следва слоевете от Раздел II: договорите са в `include/lexicon/`, реализацията в `src/`, данните в `data/`, тестовете в `tests/`. Всеки слой е отделна единица за превод с един заглавен файл, тоест зависимостите са видими в списъка с включвания и не могат да се образуват мълчаливо. Външни зависимости няма, и това е решение, а не пропуск: изискването е чужд човек да може да изгради проекта на чиста машина. Затова изпълнителят на тестовете е в самото хранилище, а базата от знания е текстов файл, четен при стартиране.

Три решения заслужават коментар. **Първо**, един и същ възел от гората може да бъде построен от повече от една продукция, и тогава завършването предлага една и съща връзка два пъти; без проверка за повторение броят на изводите нараства, без изречението да е станало по-нееднозначно. **Второ**, отхвърлянето на разчитане е нормална работа, не програмна грешка: когато моделът няма рамка за предлога, причината се предава с изключение от отделен тип, което конвейерът улавя и записва дословно, вместо да прекъсва анализа. **Трето**, нито един от двата генератора не вижда въпроса, дървото или лексикона; този на SQL цитира променлива от обхващащо ниво, което прави вложеното NOT EXISTS корелирано.

Тестовете са 30 отделни случая в пет файла, всеки регистриран в CTest поотделно, така че провалът назовава случая, а не файла. Че логическата форма е абстракция, а не преоблечен шаблон, също е тест: активният и страдателният залог пораждат побайтово еднакви заявки и на двата целеви езика. Пълният изходен текст, данните и записаните измервания са в публичното хранилище <https://github.com/Alex-Tsvetanov/lexicon>; обемът, измерен с `wc -l`, е в Табл. 1.

Таблица 1. Състав на хранилището

Път	Съдържание	Редове
<code>include/</code> , <code>src/</code>	по един договор и една реализация на слой	3345
<code>data/</code>	лексикони, граматика, схема, знания, въпроси	473
<code>apps/</code> , <code>tests/</code>	демонстрация, измерване, 30-те тестови случая	1045
<code>tools/</code> , <code>measurements/</code>	проверката на SQL и записаните измервания	236

IV. МЕТОДИКА И ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ

4.1. Какво се мери и как

Експериментът проверява три твърдения: че системата превръща вярно въпросите във формална заявка, че логическата форма е действителна абстракция, а не преоблечен шаблон над въпроса, и че перифразата съответства на построената заявка.

Наборът от 25 въпроса е замразен преди каквото и да е измерване и се чете както от демонстрацията, така и от програмата за мерене. Той съдържа 5 въпроса в активен залог към допълнението, 4 в активен залог към подлога, 3 в страдателен залог, по 2 с отрицание, с броене, с отговор да или не, с квантор и с предложно допълнение, и 3 нарочно извън обхвата, за да има покритието върху какво да се провали. Ресурсите са 56 лексеми, 80 словоформи, 25 продукции, 6 релации, 8 предложни рамки и 38 факта.

Мерките са пет: покритие на синтактичния анализ, покритие от край до край, нееднозначност (изводи, разчитания след типовата проверка и различни логически форми, отчетени поотделно, защото разликите между тях са приносят на семантичния слой), съгласуваност на генерирания SQL и време за анализ. Времената се мерят със `std::chrono::steady_clock` около самото извикване на анализатора, без еднократното зареждане на ресурсите, и се повтарят нечетен брой пъти с отчитане на медианата, защото еднократно измерване на анализ под една милисекунда мери часовника, а не анализатора.

Числата са получени на 19 август 2026 г. при превод с g++ 15.2.0 (MinGW-w64) в режим Release по стандарт C++20, CMake 4.3.2 и Ninja 1.13.2 върху Windows 11 с 12 логически процесора, а проверката на SQL с модула sqlite3 на Python 3.14. Необработеният изход е в `docs/measurements/` и е част от хранилището, тоест всяко число се проследява до реда, от който идва.

4.2. Работата на конвейера върху един въпрос

Листинг 1 показва изхода за нееднозначен въпрос, съкратен до съществените стъпки. Държавата може да принадлежи на актьорите или на филмите, моделът има релация и за двете, така че и двете разчитания оцеляват и дават различен отговор. Системата избира по-близкото прикрепване и казва кое и защо.

```
QUESTION Which films with actors from France did Nolan direct?
```

```

2. PARSE FOREST
derivations 2    forest nodes 28    chart items 111    ambiguity points 1

3. READINGS
reading 1: kept, attachment cost 4
    select x0 where Film(x0) & Person(x1) & acted_in(x1, x0)
        & film_country(x0, c_france) & directed(p_nolan, x0)
reading 2: kept, attachment cost 2
    select x0 where Film(x0) & Person(x1) & person_country(x1, c_france)
        & acted_in(x1, x0) & directed(p_nolan, x0)

4. CHOSEN READING
chose reading 2 with attachment cost 2, the closest attachment among
the surviving readings

6. SQL
SELECT DISTINCT t0.title AS answer
FROM film AS t0, person AS t1, country AS e2, person AS e3,
    acted_in AS r4, directed AS r5
WHERE e2.id = 'c_france' AND e3.id = 'p_nolan'
    AND t1.country_id = e2.id
    AND r4.person_id = t1.id AND r4.film_id = t0.id
    AND r5.person_id = e3.id AND r5.film_id = t0.id ORDER BY 1;

7. SPARQL
SELECT DISTINCT ?x0 WHERE { ?x0 a :Film . ?x1 a :Person .
    ?x1 :comesFrom :c_france . ?x1 :actedIn ?x0 .
    :p_nolan :directed ?x0 . } ORDER BY ?x0

8. PARAPHRASE BACK TO ENGLISH
You are asking for the films with actors from France that were directed
by Christopher Nolan.

9. ANSWER FROM THE BUNDLED KNOWLEDGE BASE
Inception

```

Листинг 1: Съкратен изход на lexicon за нееднозначен въпрос

4.3. Покритие и съгласуваност по слоеве

Разбивката по слоеве, Табл. 2, отнася грешката към конкретен слой, вместо да я остави обща за системата.

Таблица 2. Покритие и съгласуваност по слоеве

Слой	Мярка	Резултат
морфология	въпроси, в които всяка дума е позната	22 от 25
синтактичен анализ	въпроси с поне един извод	23 от 25
семантично изображение	въпроси с поне едно разчитане върху модела	22 от 25
генериране на SQL	заявки, които връщат същото като оценителя	22 от 22
генериране на SPARQL	произведена заявка за всеки построен израз	22 от 22
синтез	произведена перифраза за всеки построен израз	22 от 22

Съгласуваността на SQL е измерена независимо: заявката се изпълнява върху SQLite, зареден със схемата и данните от `lexicon --emit-sql-schema`, а резултатът се сравнява с отговора на оценителя, който работи направо върху логическата форма и не вижда SQL. 22 от 22 съвпадат, включително отрицанията и релационното деление, което идва от *every*. Всичките 30 теста минават. Трите въпроса с непозната дума са същите три, които са нарочно извън обхвата. Отделна мярка за точност на морфологичния анализ спрямо ръчно проверени разчитания **не е провеждана**: редът в Табл. 2 отчита само дали думите са познати, а не дали приписаните признаци са верни за произволен текст.

4.4. Поведение при нееднозначност

Шест въпроса допускат повече от един извод (Табл. 3). При долните три разчитанията се различават по дървото, но не по значението, и каноничното сравнение ги събира в едно. При горните три разчитанията са наистина различни и всяко дава различен отговор.

Таблица 3. Разчитания преди и след семантичната проверка

Въпрос	изводи	върху модела	значения
Which actors acted in every film by Villeneuve?	2	2	2
Which actors acted in no film by Nolan?	2	2	2
Which films with actors from France did Nolan direct?	2	2	2
Which actors from France acted in films by Nolan?	2	2	1
Who directed a film with Cotillard in 2010?	3	3	1
Which directors from Canada directed films in 2016?	2	2	1

Отхвърляне по типова проверка не се среща в замразения набор, но механизмът се проверява от тестовете. Върху *Which films with actors in 2010 did Nolan direct?* синтактичният слой дава 2 извода, а семантичният отхвърля единия с причината „no relation for in between Person and Year“. Този въпрос не е част от набора и числото не влиза в Табл. 2.

4.5. Нееднозначност спрямо размера на граматиката

Вторият експеримент минава същия набор през три граматики. Втората добавя две продукции към работните 25, третата още една, и никоя не описва низ, който първата не описва.

При работната граматика наборът дава 30 извода, 25 значения и 7 точки на нееднозначност. Двете допълнителни продукции вдигат изводите на 47, тоест с 57 на сто, а точките на 19, без да променят нито покритието (23 анализирани въпроса и при трите), нито броя на значенията. Третата граматика не променя нищо: добавената продукция за две съседни съществителни не се задейства, защото нито един въпрос не поставя две съществителни едно до друго. Цената на една продукция следователно не е самото ѝ съществуване, а това дали изреченията стигат до нея. Значенията остават 25 и при трите, но си струва да се каже без заобикалки: излишните разчитания отпадат, защото за тези продукции няма написано семантично правило, а не защото моделът на знанието ги е отхвърлил.

4.6. Време за анализ

Табл. 4 дава медианата и минимума от 201 повторения за изречения, всяко от които добавя по една предложна група към предходното.

Таблица 4. Време за анализ спрямо дължината на изречението, 201 повторения

групи	думи	изводи	елементи	възли	медиана, us	минимум, us
0	4	1	48	13	14.6	14.1
1	6	2	66	22	22.7	21.9
2	8	3	84	31	30.0	26.5
3	10	4	102	40	35.3	33.6
4	12	5	120	49	45.4	42.3
5	14	6	138	58	52.8	49.6
6	16	7	156	67	64.7	55.9

Изводите растат като $k + 1$ за k предложни групи, а елементите и възлите линейно, с точно 18 елемента и 9 възела на добавена група, и времето върви заедно с тях. Кубичната горна граница не се проявява в този диапазон, защото изреченията са къси, а граматиката не поражда плътна таблица. Средното време за един въпрос от набора, измерено веднъж за всеки въпрос, е 45,1 микросекунди; това число е по-шумно, защото не е повторено.

4.7. Съпоставка с базово решение и анализ на провалите

Съпоставката е проведена. Условието, което този раздел поставяше, е фиксиран модел, фиксирана версия и фиксирани настройки, защото без тях резултатът не е възпроизводим. Локален модел изпълнява това условие по-добре от услуга през чужд програмен интерфейс: отдалечената крайна точка се сменя без предупреждение и старото поведение е невъзстановимо, докато модел, изтеглен по контролна сума и пуснат при температура 0 с фиксирано зърно, се повтаря. Затова базовото решение тук е локално.

Моделът е `qwen2.5-coder:7b`, контролна сума `dae161e27b0e`, при температура 0, зърно 1 и `num_predict` 400, пуснат през Ollama 0.32.9. Програмата за мерене е `tools/llm_baseline.py`, а второто условие се получава с ключа `--with-labels`. Необработеният изход е в `docs/measurements/llm_baseline_schema_only.txt` и `docs/measurements/llm_baseline_with_labels.txt`. Повторимостта е проверена, не допусната: двете условия бяха пуснати два пъти и присъдите по всичките 25 въпроса съвпаднаха.

Критерият е равенство по изпълнение, а не по низ. Заявката на конвейера и заявката на модела се изпълняват върху вградената база и двата резултата се сравняват като неподредени множества с повторения. Две различни заявки, които връщат едни и същи редове, се броят и двете за верни; сравнение по низ би отчело верни отговори като грешни. Условието са две и разликата между тях е основният резултат (Табл. 5).

Таблица 5. Съвпадения на локалния езиков модел с конвейера, 25 въпроса

Условие	съвпадения	неизпълними заявки
само схемата	5 от 25	1
схемата и стойностите на етикетите в данните	16 от 25	2

Второто условие съществува, защото правилният конвейер получава лексикон, който свързва повърхнинните форми с обектите. Да се отнемат на модела всички етикети, докато другата страна има лексикон, не е съпоставка при равни условия. Подаването на етикетите утроява съвпаденията, от 5 на 16. Това показва къде е бил основният провал: в свързването на имената с обектите, а не в построяването на заявката. Оттам се вижда и къде стои действителната стойност на лексикона в тази работа.

Деветте разлики при подадени етикети са изброени поименно (Табл. 6). Разбивката казва повече от самото число и на две места говори срещу проекта.

Таблица 6. Деветте разлики при подадени етикети

Въпрос	Какво се разминава	Причина
Did Nolan direct Inception?	1 срещу „Yes“	критерият
Did Villeneuve direct Inception?	0 срещу „No“	критерият
Which films did Nolan direct or produce?	конвейерът не дава заявка, моделът е верен	конвейерът
Which films that Nolan directed were released in 2010?	конвейерът не дава заявка, моделът е верен	конвейерът
Which films did Kubrick direct?	конвейерът не дава заявка, моделът връща празно, което е вярно	конвейерът
Who directed Inception?	измислена колона person.person_id	моделът
Which directors from Canada directed films in 2016?	погрешно назована T3.name	моделът
Which actors from France acted in films by Nolan?	празно вместо Marion Cotillard	моделът
Which actors acted in every film by Villeneuve?	режисьорът вместо актьора	моделът

Два от деветте не са провал на модела. При *Did Nolan direct Inception?* и *Did Villeneuve direct Inception?* конвейерът връща целите числа 1 и 0, а моделът низовете „Yes“ и „No“. Значението е едно и също, но множествата от редове се различават. Това е ограничение на критерия и се записва като такова; критерият не се разхлабва след като числата са известни.

Три от деветте са там, където конвейерът изобщо не произвежда SQL. Моделът отговаря вярно на *Which films did Nolan direct or produce?* и *Which films that Nolan directed were released in 2010?*, а на *Which films did Kubrick direct?* връща празно множество, което е вярно, защото Кубрик не е в данните. Тоест моделът успява точно там, където правилната система се проваля.

Четири са действителни провали на модела. При *Who directed Inception?* и *Which directors from Canada directed films in 2016?* той измисля колони, които не съществуват, и заявката не се изпълнява. При *Which actors from France acted in films by Nolan?* връща празно множество, а отговорът е Marion Cotillard, при съединение през две стъпки с филтър по държава. При *Which actors acted in every film by Villeneuve?* връща режисьора вместо актьора, тоест се проваля на квантора; обхватът на кванторите вече стои в по-нататъшната

работа.

Тоест от 22-те въпроса, на които конвейерът отговаря, моделът съвпада по резултат на 16, на още два дава семантично същия отговор в друг запис, и на четири се проваля действително.

Границите се казват, а не се подразбират. Моделът е достатъчно малък, за да се побере в 6 GB видеопамет. Числата казват какво прави един малък квантуван локален модел върху 25 въпроса срещу ръчно написан каталог от филми, и нищо за челен модел или за езиковите модели изобщо.

Провалите на конвейера са три и вече се появиха по име. Първите два се провалят в синтактичния слой, защото граматиката няма съчинение и няма относително изречение, а третият в семантичния, защото името не е в лексикона. Първите два се поправят с продукция и семантично правило, третият с един ред данни. Нито един провал не идва от логическата форма или от генераторите. Покритието на граматиката следователно е по-скъпото ограничение, и всяко негово разширение носи нова нееднозначност с измерената по-горе цена. Че точно на тези три въпроса базовото решение отговаря вярно, дава мащаб на тази цена.

ЗАКЛЮЧЕНИЕ

Проектът изгражда интерфейс на естествен език, който превръща въпрос от обикновената реч във формална заявка чрез морфологичен, синтактичен и семантичен анализ, и връща перифраза на разбрания въпрос, преди заявката да бъде изпълнена. Всички слоеве са символни, което прави всяко тълкуване проследимо назад до думите, които са го породили.

Предимства. Обяснимост на всяка стъпка, включително на отказа: причината за отхвърлено разчитане е изречение, което стига до потребителя дословно. Потвърждение преди изпълнение, с перифраза, построена от логическата форма, а не от въпроса. Общ ресурс между анализа и синтеза. Едно семантично представяне, от което излизат два целеви езика без промяна в предходните слоеве. Липса на външни зависимости.

Недостатъци. Покритието е ограничено от лексикона и от граматиката, а пренасянето към нова предметна област изисква ръчно описание на знанието. Моделът няма подтипове, така че актьор и режисьор са едно и също нещо. Обхватът на кванторите е фиксиран, а не изчислен. Перифразата разделя съседните определения с „и“, което не звучи естествено.

Резултати. От 25 въпроса 23 получават поне един извод и 22 достигат до заявка. 22 от 22 генерирани SQL заявки връщат същото като независимия оценител. Шест въпроса допускат повече от един извод; при три разчитанията се събират в едно значение, при три остават различни и системата съобщава кое е избрала и защо. Времето за анализ расте от 14,6 до 64,7 микросекунди при увеличаване на изречението от 4 на 16 думи. Две продукции, които не разширяват покритието, увеличават изводите с 57 на сто, без да добавят ново значение. Трите провала са изброени поименно: два в граматиката, един в лексикона, нито един в логическата форма или в генераторите. Съпоставката с базово решение върху езиков модел е проведена при фиксиран локален модел qwen2.5-coder:7b, температура 0 и зърно 1, с критерий равенство по изпълнение: 5 от 25 съвпадения само по схемата и 16 от 25 при подадени и етикетите. Основният провал на модела е свързването на имената с обектите, а не построяването на заявката, а на трите въпроса без заявка от конвейера той отговаря вярно.

По-нататъшна работа. Съчинение и относително изречение в граматиката, което покрива двата структурни провала, но носи нова нееднозначност с измерената

в подраздел 4.5 цена. Подтипове в модела на знанието. Изчисляване на обхвата на кванторите. Пренасяне на собствените имена направо от базата от знания, за да не се дублира лексиконът. И повтаряне на съпоставката с по-голям модел и по-голям набор от въпроси, защото проведената мери един малък локален модел върху 25 въпроса.

ЛИТЕРАТУРА

- [1] N. Chomsky, *Syntactic Structures*. The Hague: Mouton, 1957.
- [2] J. Earley, “An efficient context-free parsing algorithm,” *Communications of the ACM*, vol. 13, no. 2, pp. 94–102, 1970.
- [3] D. H. Younger, “Recognition and parsing of context-free languages in time n^3 ,” *Information and Control*, vol. 10, no. 2, pp. 189–208, 1967.
- [4] K. R. Beesley and L. Karttunen, *Finite State Morphology*. Stanford, CA: CSLI Publications, 2003.
- [5] W. A. Woods, “Transition network grammars for natural language analysis,” *Communications of the ACM*, vol. 13, no. 10, pp. 591–606, 1970.
- [6] A. Tsvetanov and I. Stankov, “Matching text from start to finish against multiple regular expressions,” in *2024 32nd National Conference with International Participation (TELECOM)*, (Sofia, Bulgaria), IEEE, 2024.
- [7] I. Androutsopoulos, G. D. Ritchie, and P. Thanisch, “Natural language interfaces to databases: An introduction,” *Natural Language Engineering*, vol. 1, no. 1, pp. 29–81, 1995.
- [8] J. M. Zelle and R. J. Mooney, “Learning to parse database queries using inductive logic programming,” in *Proceedings of the Thirteenth National Conference on Artificial Intelligence (AAAI-96)*, pp. 1050–1055, 1996.
- [9] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, “Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3911–3921, 2018.
- [10] W3C SPARQL Working Group, “SPARQL 1.1 query language.” W3C Recommendation, 2013. <https://www.w3.org/TR/sparql11-query/>.
- [11] J. F. Sowa, *Knowledge Representation: Logical, Philosophical, and Computational Foundations*. Pacific Grove, CA: Brooks/Cole, 2000.